arXiv:2311.05884v1 [cs.IR] 10 Nov 2023

Hiformer: Heterogeneous Feature Interactions Learning with Transformers for Recommender Systems

Huan Gui
Google DeepMind
Mountain View, California, USA
hgui@google.com

Ruoxi Wang
Google DeepMind
Mountain View, California, USA
ruoxi@google.com

Ke Yin
Google Inc
Mountain View, California, USA
keyin@google.com

Long Jin
Google Inc
Mountain View, California, USA
longjin@google.com

Maciej Kula
Google DeepMind
Mountain View, California, USA
maciejkula@google.com

Taibai Xu
Google Inc
Mountain View, California, USA
taibaixu@google.com

Lichan Hong
Google DeepMind
Mountain View, California, USA
lichan@google.com

Ed H. Chi
Google DeepMind
Mountain View, California, USA
edchi@google.com

ABSTRACT

Learning feature interaction is the critical backbone to building recommender systems. In web-scale applications, learning feature interaction is extremely challenging due to the sparse and large input feature space; meanwhile, manually crafting effective feature interactions is infeasible because of the exponential solution space. We propose to leverage a Transformer-based architecture with attention layers to automatically capture feature interactions. Transformer architectures have witnessed great success in many domains, such as natural language processing and computer vision. However, there has not been much adoption of Transformer architecture for feature interaction modeling in industry. We aim at closing the gap. We identify two key challenges for applying the vanilla Transformer architecture to web-scale recommender systems: (1) Transformer architecture fails to capture the heterogeneous feature interactions in the self-attention layer; (2) The serving latency of Transformer architecture might be too high to be deployed in web-scale recommender systems. We first propose a heterogeneous self-attention layer, which is a simple yet effective modification to the self-attention layer in Transformer, to take into account the heterogeneity of feature interactions. We then introduce Hiformer (**H**eterogeneous **I**nteraction Trans**former**) to further improve the model expressiveness. With low-rank approximation and model pruning, Hiformer enjoys fast inference for online deployment. Extensive offline experiment results corroborates the effectiveness and efficiency of the Hiformer model. We have successfully deployed the Hiformer model to a real world large scale App ranking model at Google Play, with significant improvement in key engagement metrics (up to +2.66%).

KEYWORDS

Heterogeneous Feature Interaction, Transformer, Recommender System

ACM Reference Format:
Huan Gui, Ruoxi Wang, Ke Yin, Long Jin, Maciej Kula, Taibai Xu, Lichan Hong, and Ed H. Chi. 2023. Hiformer: Heterogeneous Feature Interactions Learning with Transformers for Recommender Systems. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.475/123_4

1 INTRODUCTION

The internet is inundated with a plethora of information, making it challenging for users to effectively navigate and locate relevant content. Recommender systems filter information and present the most relevant content to individual users [35, 49]. It is common to formulate recommender systems as a supervised machine learning problem, with the goal of increasing user's positive engagements with the recommendations, such as clicks [40, 45, 46], watches [50], or purchases [36]. Therefore, the deployment of recommender systems with high prediction accuracy is of paramount importance as it could have a direct influence on business' financial performance, such as sales and revenue [3–5, 49].

Feature interactions are where multiple features have complex collaborative effects on the prediction of an outcome [43, 46]. It is one of the crucial components of recommender system. For example, it's observed that users frequently download food delivery apps during meal times [8], indicating the feature interactions of app ids (*e.g.* food delivery apps) and temporal context (*e.g.* meal times) can provide vital signal for predicting user behavior and making personalized recommendations. However, modeling feature interactions in web-scale recommender systems presents three significant challenges. First of all, the right feature interactions are often domain-specific, manually crafting the feature interactions is time-consuming and requires domain knowledge. Secondly, for

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2023 Association for Computing Machinery.
ACM ISBN 123-4567-24-567/08/06…$15.00
https://doi.org/10.475/123_4

web-scale applications, due to the large number of raw features, the search of possible feature interactions has an exponential solution space. This makes it infeasible to manually extract all the interactions. Last but not least, the extracted feature interactions may not generalize well to other tasks. Due to its importance, feature interaction continues to attract increasing attention from both academic and industry [4, 5, 27, 33, 49].

In recent years, deep neural networks (DNNs) have garnered significant attention in various research fields, due to their exceptional model performance as well as their ability in representation learning [22, 49]. DNNs have been widely adopted for recommender system for sparse input features representation learning [30] and feature interaction learning [1, 3, 8, 26, 29, 32, 40, 45, 46, 49]. It has become a powerful tool to map the large and sparse input into a low-dimensional semantic space. Some of the recent works [8, 24, 29, 32, 46] in feature interaction learning are based on explicit feature interaction functions, such as Factorization Machine [26, 34], inner product [32], kernel Factorization Machine [33], and Cross Network [45, 46]. DNNs are also leveraged to learn the implicit feature interactions with multiple layers of artificial neurons with non-linear activation functions [3, 29].

Another line of work is based on the attention mechanism, in particular, the Transformer-based architecture [44] for feature interaction learning. The Transformer architecture has become the state-of-the-art (SOTA) model for a variety of tasks, including computer vision (CV) [9, 21] and natural language processing (NLP) [19]. AutoInt [40] and InterHAt [25] proposed to leverage the multi-head self-attention layer for feature interaction learning. However, unlike the NLP field, the feature semantics in recommender systems are dynamic under different contexts.

Suppose we are recommending food delivery apps to users and we have the features: `app_id`, `hour_of_day`, and `user_country`. In this case, `app_id` could mean one thing for `hour_of_day` and another for `user_country`. To effectively interact different features, we would need semantic awareness and semantic space alignment. Transformer models unfortunately do not consider this. In the vanilla attention layer, the features are projected through the same projection matrices (*i.e.,* $\mathbf{W_Q}$ and $\mathbf{W_k}$) that are shared across all features. This is a natural design for applications where the feature (text token) semantics are independent of the context; however for recommender systems where the feature semantic are often dependent of the context, this homogeneous design would lead to limited model expressiveness for feature interaction learning.

Besides the limitation of not being able to capture heterogeneous feature interactions, another limitation of Transformer models is on latency. In web-scale applications, there are frequently a vast number of users and requests. It is crucial that the models are capable of managing an extremely high number of queries per second, to deliver an optimal user experience. However, the inference cost of Transformer architectures scales quadratically with the input length, which makes real-time inference using Transformer architectures prohibitive in web-scale recommender systems.

Despite the limitations, we deem it important to study and improve Transformer-based architecture for feature interaction. First of all, due to the success of the Transformer architecture in many domains, a significant number of advancements and modifications have been made to the architecture to improve its performance and capabilities. Enabling Transformer-based feature interaction models for recommender system can serve as a bridge, to allow for the recent advancements in Transformer architectures to be applied in the recommendation domain. Secondly, with its widespread adoption, the hardware design (e.g., chip design [28]) might favor Transformer-like architectures. Hence, recommender systems with Transformer-based architectures could also enjoy the optimizations brought by hardware. Thirdly, as Transformer models for feature interaction learning are based on the attention mechanism, they provide good model explainability [40]. Building recommender models with attention mechanism in web-scale applications might open up new research opportunities.

In this paper, we consider the limitations of vanilla Transformer model architectures and propose a new model: Hiformer (**H**eterogeneous Feature **I**nteraction Trans**former**) for feature interaction learning in recommender systems. Hiformer is feature semantic aware and, more importantly, efficient for online deployment. To provide feature semantic awareness in feature interaction learning, we design a novel heterogeneous attention layer. To comply with the stringent latency requirements of web-scale applications, we leverage low-rank approximation and model pruning techniques to improve the efficiency.

We use a large-scale App ranking model at Google Play as a case study. We perform both offline and online experiments with Hiformer to corroborate its effectiveness in feature interaction modeling and efficiency in serving. To the best of our knowledge, we show for the first time that a Transformer-based architecture (i.e., Hiformer) can outperform SOTA recommendation models for feature interaction learning. We have successfully deployed Hiformer as the production model.

To summarize, the contributions of the paper are as follows:

- We propose the Hiformer model with a novel heterogeneous attention layer to capture the complex collaborative effects of features in feature interaction learning. Compared with existing vanilla Transformer based approaches, our model has more model expressiveness.
- We leverage low-rank approximation and model pruning to reduce the serving latency of the Hiformer model, without compromising the model quality.
- We conduct extensive offline comparison with a web-scale dataset to demonstrate the importance of capturing heterogeneous feature interactions. We show that hiformer, as a Transformer-based architecture, can outperform SOTA recommendation models.
- We perform online A/B testing to measure the impact of different models on key engagement metrics, and the Hiformer model shows significant online gain compared with the baseline model with limited latency increase.

2 PROBLEM DEFINITION

The goal of recommender system is to improve users' positive engagement, such as clicks, conversion, etc. Similar to previous studies [26, 27], we formulate it as a supervised machine learning task for engagement predictions.

Definition 2.1 (Recommender System). Let $\mathbf{x} \in \mathbb{R}^{d_x}$ denote the features of a (user, item) pair. There are categorical features $\mathbf{x}^C$

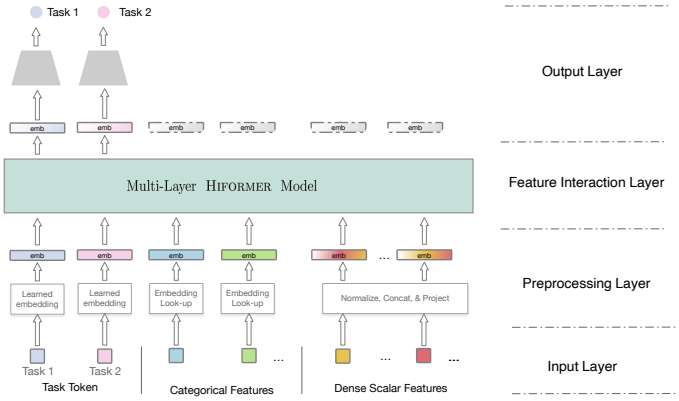


Figure 1: There are four components in the framework: Input Layer, Preprocessing Layer, Feature Interaction layer, and Output Layer. We leverage a novel HIFORMER model for heterogeneous feature interaction learning.

and dense features $\mathbf{x}^{\mathcal{D}}$. Categorical features are represented with one-hot encoding. The dense features can be considered as special embedding features with embedding dimension being 1. The numbers of categorical and dense features are $|C|$ and $|\mathcal{D}|$ respectively. The problem of recommender system is to predict if the user will engage with the item based on input features $\mathbf{x}$.

As aforementioned, feature interaction learning enables the model to capture more complex relationships among features, thereby provides virtual information for more accurate recommendations. We formally define heterogeneous feature interaction learning.

Definition 2.2 (Heterogeneous z-order Feature Interaction). Given input with $|\mathcal{F}|$ features and the input as $\mathbf{x} = \{\mathbf{x}_i\}_{i=1}^{|\mathcal{F}|}$, a z-order feature interaction is to learn a unique *non-additive* mapping function $\rho^{\mathcal{Z}}(\cdot)$ that map from the feature list $\mathcal{Z}$ with z features (*i.e.,* $[\mathbf{x}_{i_1}, \cdots \mathbf{x}_{i_z}]$) to a new representation or a scalar to capture the complex relationships among the the feature list $\mathcal{Z}$. When z = 2, $\rho^{\mathcal{Z}}(\cdot)$ captures the second order feature interactions.

For example, $g([\mathbf{x}_1, \mathbf{x}_2]) = w_1\mathbf{x}_1 + w_2\mathbf{x}_2$ is not a 2-order feature interaction, as $w_1\mathbf{x}_1 + w_2\mathbf{x}_2$ is additive. Meanwhile, $g([\mathbf{x}_1, \mathbf{x}_2]) = \mathbf{x_1}^T\mathbf{x_2}$ is a 2-order feature interaction; and $\rho^{1,2}(\cdot) = \rho^{2,1}(\cdot) = \mathbf{x}_1^T\mathbf{x}_2$, meaning the heterogeneous feature interaction is symmetric.

Before diving into model details, we define our notations. Scalars are denoted by lower case letters ($a, b, \ldots$), vectors by bold lower case letters ($\mathbf{a}, \mathbf{b}, \ldots$), matrices by bold upper case letters ($\mathbf{A}, \mathbf{B}, \ldots$).

3 MODEL

We will first present the overall framework in Section 3.1. We then propose a heterogeneous attention layer in Section 3.3, based on which we further introduce the HIFORMER model in Section 3.4.

3.1 Overview

3.1.1 Input Layer. The input layer includes the categorical features, and dense features. Additionally, we have the task embedding. The task embedding can be regarded as the CLS token [6]. The proposed framework can also be easily extended to multi-task [2] with multiple task embeddings, and each task embedding represents information of the corresponding training objective. The task embeddings in the input layers are model parameters and are learned through the end-to-end model training. We denote the number of task embeddings as t.

3.1.2 Preprocessing Layer. We have the preprocessing layer to transform the input layer into a list of feature embeddings, in addition to the task embeddings, as input to the feature interaction layers. We have one preprocessing layer per feature type.

Categorical Features. The input categorical features are often sparse and of high dimensions. Directly using the one-hot encoding for model training would easily lead to model overfitting. Therefore, a common strategy is to project the categorical features into a low-dimension dense space. In particular, we learn a projection matrix $\mathbf{W}_i^C \in \mathbb{R}^{V_i \times d}$ related to each categorical feature, and

$$\mathbf{e}_i = \mathbf{x}_i^C \mathbf{W}_i^C,$$

where $\mathbf{e}_i \in \mathbb{R}^d$, $\mathbf{x}_i^C \in \{0,1\}^{V_i}$, V_i is dimensions of the one-hot encoding for feature $\mathbf{x}_i^C$, and d is the model dimension. We will then use the dense embedded vector $\mathbf{e}_i$ as the representation of the categorical feature x_i^C. $\{\mathbf{W}_i^C\}^{|C|}$ are the model parameters and referred to as embedding look-up tables.

Dense Scalar Features. The dense scalar features are numerical features. As the dense scalar features are potentially from very different distributions, we will need to transform the features to a similar distribution (*i.e.,* uniform distribution) to ensure numerical stability [51]. Additionally, in web-scale applications, there are a large number of dense scalar features, with much less compared with the categorical features. Therefore, we aggregate all the dense scalar features and project them into $n^{\mathcal{D}}$ embeddings [29], where $n^{\mathcal{D}}$ is a hyper-parameter chosen by empirical results, $n^{\mathcal{D}} \ll |\mathcal{D}|$ to reduce the total number of features. The projection function $f_{\mathcal{D}}(\cdot)$ could be a multilayer perceptron (MLP) with nonlinear activations:

$$\mathbf{e}_i = \text{split}_i\left(f_{\mathcal{D}}\left(\text{concat}\left(\text{normalize}(\{x_i^{\mathcal{D}}\})\right)\right), \texttt{split_size} = d\right),$$

where $\mathbf{e}_i \in \mathbb{R}^d$, $\text{concat}(\cdot)$ is the concatenation function, $\text{normalize}(\cdot)$ is the feature transformation function, and $\text{split}_i(\cdot, \texttt{split_size})$ is a function to split the input tensor into tensors with equal dimension of `split_size` and takes the i-th tensor as output. In Transformer-based architectures, the inference cost scales quadratically with the length of input embedding list. By aggregating $|\mathcal{D}|$ dense scalar features into $n^{\mathcal{D}}$ embeddings, the length of feature embeddings is reduced, and as a consequence, the inference cost is also reduced, albeit with a trade-off of adding $f_D(\cdot)$.

To summarize on preprocessing layers, we transform the $|C|$ categorical features, $|\mathcal{D}|$ dense scalar features, and the t task embeddings into a list of embeddings with the model dimension d. The number of output embeddings from preprocessing layers is $L = |C| + n^{\mathcal{D}} + T$. For ease of discussion, the task embeddings are also regarded as special features in the embedding list. Therefore, feature interactions refers to not only interaction among features, but also between features and tasks.

3.1.3 Feature Interaction Layer. The feature interaction layer is used to learn the complex collaborative effects among features,

which is crucial to capture users' preferences under different contexts. The input to the feature interaction layer is the output embedding list of preprocessing layers.

3.1.4 Output Layer and Training Objectives. We only use the encoded task embeddings from the feature interaction layer for task prediction. We train a MLP tower to project the encoded task embedding to final predictions. The training tasks can be flexibly configured based on the recommendation application. For example, the training task can be a classification task if the labels are click [39, 40], installation [15, 47], or purchase [23, 31], and may also be a regression task, such as in the case of watch time [50]. Without loss of generality, we consider classification task, with a value of 0 indicating negative engagement and 1 indicating positive engagement. Thus, the training objective can be formulated as a binary classification problem with the Binary Cross Entropy (*i.e.*, Log Loss):

$$\ell = \frac{1}{N}\sum_{i}^{N} -y_i \log(p_i) - (1-y_i)\log(1-p_i),$$

where y_i and p_i are the ground truth label and predicted probability of engagements for example i respectively, and N is the total number of training examples.

3.2 Heterogeneous Feature Interactions

The Transformer architecture has become the de-facto standard for many tasks such as NLP, CV, etc., and multi-head self-attention layer has achieved remarkable performance in modeling complicated relations in sequences [9, 19, 21, 44]. Nevertheless, the Transformer architecture has not been extensively applied to web-scale recommender systems. It has been observed that Transformer-based architectures, such as AutoInt [40], may not demonstrate optimal performance when applied to web-scale recommender systems.

Recall that the attention score computation of vanilla Transformer, the parameterization of feature interactions learning are shared across all feature groups in the multi-head self-attention layer. This parameter sharing design comes naturally in NLP tasks, as the input to a Transformer in NLP tasks is usually a sentence represented by a list of text embeddings. Text embeddings primarily encode information that is context independent. We refer this setup as *homogeneous* feature interactions. The detailed illustration of the Transformer architecture is shown in Figure 4(a).

By sharp contrast, there is diverse contextual information in recommender systems. The learned embedding representations are more dynamic and encompass information from multiple perspectives. In the food delivery app recommendation example, we have three features: `app_id` (*e.g.*, Zomato), `hour_of_day` (*e.g.*, 12pm), and `user_country` (*e.g.*, India). The learned embedding representation for the app Zomato encodes various implicit information, including the nature of the app, the countries in which it is popular, the times at which it is frequently used, and the language of its users, etc. Not all the encoded information would be useful during certain feature interaction learning. When learning the feature interaction between `app_id` and `user_country`, likely the information of the countries in which the app is popular is much more important than the others. Thus, the core of heterogeneous feature interaction learning is to provide contextual awareness for information selection and transformation, to achieve more accurate feature interaction learning.

3.3 Heterogeneous Attention Layer

We first introduce a simple but effective modification to the multi-head self-attention layer in the Transformer model: the heterogeneous multi-head self-attention layer, to capture the heterogeneous second-order feature interactions. For simplicity, we refer to the heterogeneous multi-head self-attention layer as the heterogeneous attention layer, and the corresponding model as heterogeneous attention model. In particular, we redesign the multi-head self-attention score computation for feature i and j:

$$\text{Att}(i,j)^h = \frac{\exp\big(\phi^h_{i,j}(\mathbf{e}_i,\mathbf{e}_j)\big)}{\sum_{m=1}^{L}\exp\big(\phi^h_{i,m}(\mathbf{e}_i,\mathbf{e}_m)\big)}, \tag{1}$$

where h refers to the h-th head among total H heads, L is the embedding list length, $\phi^h_{i,j}(\mathbf{e}_i,\mathbf{e}_j)$ measures semantic correlation between embedding $\mathbf{e}_i$ and $\mathbf{e}_j$ regarding head h. We will then refer the vanilla self-attention layer as homogeneous attention layer.

For every feature pair (i,j), we have a unique $\phi^h_{i,j}(\cdot,\cdot)$ to measure semantic correlation. In other words, $\phi^h_{i,j}(\cdot,\cdot)$ serves the purpose of selecting the most relevant information from $\mathbf{e}_i$ and $\mathbf{e}_j$ for the context of feature pair (i,j). We can have arbitrary functions for $\phi^h_{i,j}(\cdot,\cdot) : [\mathbb{R}^d, \mathbb{R}^d] \to \mathbb{R}$, such as some explicit non-additive functions, a neural network with nonlinear transformation. Due to its simplicity, we opt for the dot-product function. Accordingly,

$$\phi^h_{i,j}(\mathbf{e}_i,\mathbf{e}_j) = \frac{\mathbf{e}_i\mathbf{Q}^h_i(\mathbf{e}_j\mathbf{K}^h_j)^T}{\sqrt{d_k}}, \tag{2}$$

where $\mathbf{Q}_i \in \mathbb{R}^{d\times d_k}$, $\mathbf{K}_j \in \mathbb{R}^{d\times d_k}$ are the query and key projections for feature j and j respectively, and $\sqrt{d_k}$ is to normalize magnitude of the dot product, which is often set to be $d_k = d/H$.

With the attention weights computed in Eq (1), we can then compute the output of the heterogeneous attention layer as follows:

$$\mathbf{o}_i = \text{concat}\Big(\big\{\sum_j \text{Att}(i,j)^h\mathbf{e}_j\mathbf{V}^h_j\big\}_{h=1}^{H}\Big)\mathbf{O}_j, \tag{3}$$

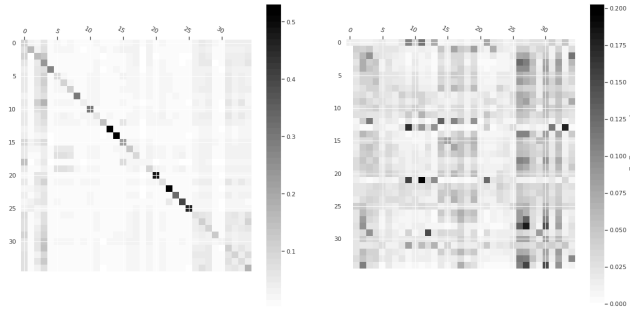
where $\mathbf{V}_j \in \mathbb{R}^{d\times d_v}$, $\mathbf{O}_j \in \mathbb{R}^{Hd_v\times d}$ as value and output projections, and d_v is frequently set as $d_v = d/H$.

Similar to the heterogeneous attention layer, we also design a heterogeneous fully connected Feed Forward Net (FFN) per feature. The FFN is implemented through two linear transformation function with Gaussian Error Linear Unit (GELU) activation [12, 37]:

$$\text{FFN}^i_{\text{GELU}}(\mathbf{o}_i) = \text{GELU}(\mathbf{o}_i\mathbf{W}^i_1 + \mathbf{b}^i_1)\mathbf{W}^i_2 + \mathbf{b}^i_2, \tag{4}$$

where $\mathbf{W}^i_1 \in \mathbb{R}^{d\times d_f}$, $\mathbf{W}^i_2 \in \mathbb{R}^{d_f\times d}$, $\mathbf{b}^i_1 \in \mathbb{R}^{d_f}$, $\mathbf{b}^i_2 \in \mathbb{R}^d$, and d_f is the layer size of the intermediate layer. Following existing studies [44], d_f is set as $d_f = 4d$, d is the dimensionality of model.

As we can see the key change of the heterogeneous attention layer, compared with self-attention layer is that we learn individual query, key, and value (QKV) projection matrices (*i.e.*, $\mathbf{Q}, \mathbf{K}, \mathbf{V}$) per feature. Therefore, we increase the number of parameters in the heterogeneous attention layer, compared with the vanilla Transformer model. The number of parameters scales linearly with the



(a) Vanilla Transformer Attention Layer. (b) Heterogeneous Attention Layer.

Figure 2: Attention patterns with feature interaction layers.

length of input embedding list. However, it is worth pointing out that the total numbers of FLOPs are the same for the heterogeneous attention layer and homogeneous one (the standard multi-head self-attention layer in the Transformer). This is because we have exactly the same ops compared with homogeneous Transformer layer, but with different parameterizations.

We visualize the learned attention pattern $\text{Att}(\cdot,\cdot)$ in Figure 2. The attention pattern learned with the vanilla Transformer model, in Figure 2(a), is relatively sparse and exhibits a strong diagonal pattern, indicating that the attention layer might not effectively capture the collaborative effects between different features. Meanwhile, for the heterogeneous attention layer, there is a dense attention pattern. Intuitively, the observation could be explained by the fact that the heterogeneous attention layer might be better at capturing the collaborative effects among features thanks to the transformation matrix for feature semantic alignment.

3.4 Hiformer

Now we are ready to introduce the Hiformer model, where we further increase the model expressiveness, by introducing composite feature learning in the QKV projections. Take key projection as an example. Instead of learning key feature projection per feature, we redefine the key projection as follows:

$$[\hat{\mathbf{k}}_1^h, \dots \hat{\mathbf{k}}_L^h] = \text{concat}([\mathbf{e}_1^h, \dots \mathbf{e}_L^h])\hat{\mathbf{K}}^h, \tag{5}$$

where $\hat{\mathbf{K}}^h \in \mathbb{R}^{Ld \times Ld_k}$. Compared with key projection matrices in Section 3.3, $[K_i^h, \dots, K_L^h] \in \mathbb{R}^{d \times Ld_k}$, we increase model expressiveness by adding more parameters to key projections. We call the new projection $\hat{\mathbf{K}}^h$ as Composite projection. Mathematically, instead of learning feature interactions explicitly with feature pairs, we first transform the feature embedding list into composite features as keys and values; then we learn heterogeneous feature interaction between composite features and task embeddings. Similarly, we apply cross projection to query and value projections.

Similarly as in Eq (1), the Hiformer model computes the attention scores between the query $\mathbf{q}_i$ and and key $\mathbf{k}_i$ as follows:

$$\text{Att}_{\text{Composite}}(i,j)^h = \frac{\exp(\hat{\mathbf{q}}_i^h (\hat{\mathbf{k}}_i^h)^T / \sqrt{d_k})}{\sum_l^L \exp(\hat{\mathbf{q}}_i^h (\hat{\mathbf{k}}_l^h)^T / \sqrt{d_k})},$$

With the attention score, we can then compute the output of attention layer in Hiformer model as:

$$\hat{\mathbf{o}}_i = \text{concat}\left(\{\sum_m \text{Att}^h_{\text{Composite}}(i,j)\hat{\mathbf{v}}_i^h\}_{h=1}^H\right)\mathbf{O}_j. \tag{6}$$

3.5 Better Efficiency for Hiformer

Compared with the Transformer model and the heterogeneous attention layer, the Hiformer model comes with more model expressiveness, but also computation cost during inference. Due to the introduced Composite attention layer, the existing efficient Transformer architectures [42] cannot be directly applied.

The length of the feature embedding list is $L = |C| + n^{\mathcal{D}} + t$. With the model dimension as d, we have inference cost breakdown [1] for Hiformer: QKV projection: $3L^2d^2$, attention score computation: $2L^2d$, output projection: Ld^2, FFN network: $8Ld^2$. As a result, total computation is $O(L^2d^2 + L^2d + Ld^2)$. To deploy Hiformer for real-time inference, we reduce the inference cost of Hiformer through low-rank approximation and pruning.

3.5.1 Low-rank approximation. As we can see, the outstanding term for Hiformer inference cost is QKV projections with computation complexity of $O(L^2d^2)$.

With Composite key projection defined in (5), we can approximate $\hat{\mathbf{K}}^h$ with low-rank approximation, i.e.,

$$\hat{\mathbf{K}}^{\mathbf{h}} = \mathbf{L}_k^h (\mathbf{R}_k^h)^T, \tag{7}$$

where $\mathbf{L}_v^h \in \mathbb{R}^{Ld \times r_v}$, $\mathbf{R}_v^h \in \mathbb{R}^{Ld_k \times r_v}$, and r_v is the rank of the low-rank approximation for $\hat{\mathbf{K}}^h$. Low-rank approximation reduces the computation cost for value projection to $O(Lr_v(d + d_k))$. Similarly, it can be applied to query and value composite projection with rank of r_k and r_v. If $r_k < Ld_k/2$ and $r_v < Ld_v/2$, the cost will be reduced. The low-rank structure is also observed in data analysis. Taking $\hat{\mathbf{V}}^h$ as an example, we plot the singular values in Figure 3, where we can see that the $\hat{\mathbf{V}}^h$ matrix is indeed low-rank.

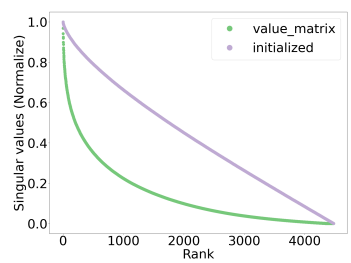
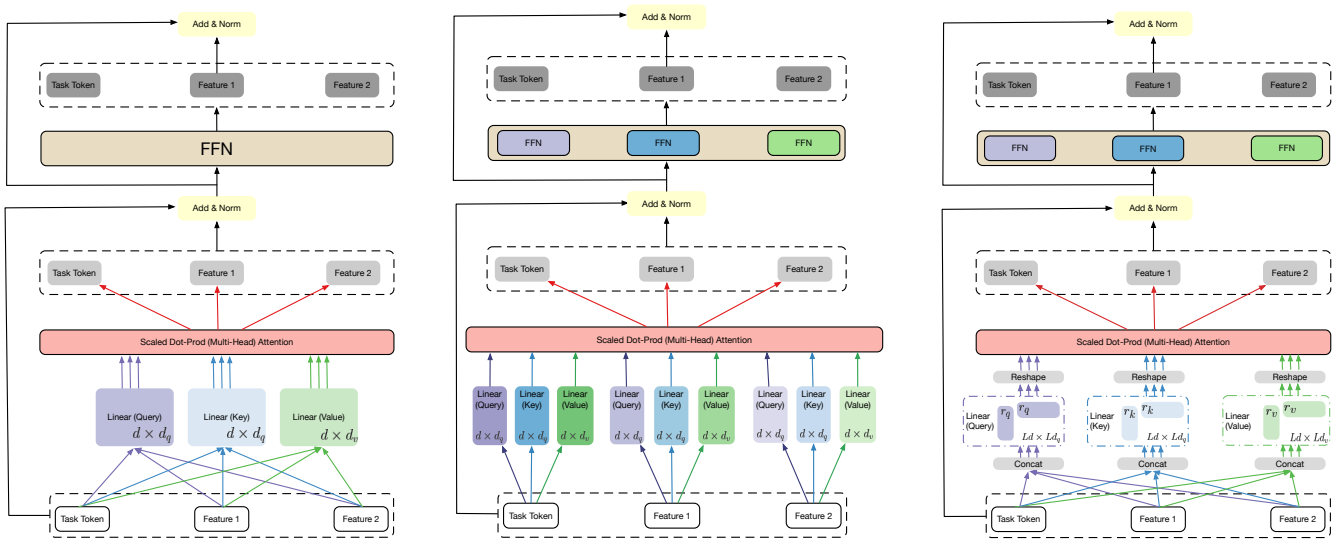


Figure 3: Singular values for $\hat{\mathbf{V}}^{\mathbf{h}}$ in Composite projection.

After applying the low-rank approximation, the Hiformer model complexity is computed as follows: the query and key projection: $O(Lr_k(d + d_k))$, value projection $O(Lr_v(d + d_v))$, attention layer $O(L^2d)$, output projection $O(Ld^2)$, and FFN layer $O(Ld^2)$. With $d_k = d/H$, $d_v = d/H$, and $r_k < d_k, r_v < d_v$, we have the computation complexity as $O(L^2d + Ld^2)$, which scales quadratically with the length of the input embedding list L.

[1] This computation is based on the setting where $d_k = d/H$, $d_v = d/H$, and $d_f = 4d$.



(a) Vanilla Transformer. (b) Transformer with Heterogeneous Attention. (c) Hiformer.

Figure 4: Illustration on capturing feature interaction with different Transformer-based architectures. with one task token and two feature tokens. In 4(a), apply vanilla Transformer for feature interaction learning. In 4(b), we explicitly model the heterogeneous feature interaction through a heterogeneous interaction layer. In 4(c), we propose the Hiformer model, which further improves the model expressiveness compared with the heterogeneous attention layer in 4(b). d is hidden layer size, d_q, and d_v are Query and Value dimensions, L is the total input length, r_* corresponding to the rank of the corresponding QKV projections (see Eq (7)).

3.5.2 Model Pruning. Recall that in the Output Layer (see Figure 1), we are only using the encoded task embeddings for final objective training and predictions. Therefore, we can save some computation in the *last layer* of the Hiformer model by pruning the computation of encoding feature embeddings. In particular, the pruning translates to only using task embeddings as query, and using both task embedding and feature embeddings as key and value in the Hiformer attention layer. Therefore, we have computation complexity of the last Hiformer layer: query projection: $O(tr_k(d+d_k))$, key projection: $O(Lr_k(d+d_k))$, value projection: $O(Lr_v(d+d_v))$, attention layer computation: $O(Ltd)$, output projection: $O(td^2)$, FFN layer: $O(td^2)$. Suppose the number of task t ($\ll L$) is considered to be a constant. We have the computation complexity of the last layer of Hiformer as $O(L(r_k+r_v)d+Ltd+td^2)$.

Pruning enables us to reduce the computation complexity of the last Hiformer layer from scaling quadratically to scaling linearly with L. This pruning technique can be applied to all Transformer-based architectures. Some of the recent work , such as Perceiver [14], has also introduced a similar idea to reduce the serving cost of Transformer models from scaling quadratically to linearly.

4 EXPERIMENT

We undertake a thorough empirical examination of the proposed model architectures with the web-scale App ranking model at Google Play. We conduct both offline and online experiments, with which we aim at answering the following research questions:

- **Q1.** Is the heterogeneous attention layer able to capture the heterogeneous feature interactions to give better recommendations, compared with vanilla Transformer models?
- **Q2.** With more model expressiveness in the Hiformer model, could it further improve model quality?
- **Q3.** How is the serving efficiency of the Hiformer model compared with SOTA models?
- **Q4.** How do the hyper-parameter settings impact the model performance and serving latency and how to make the performance and efficiency trade-offs?

4.1 Offline Evaluation Setup

4.1.1 Dataset. For offline evaluation, we use the logged data from the ranking model. Users' engagements are training labels, either 0 or 1 indicating if the engagement happens between user and the app and the training loss is LogLoss (see Eq (3.1.4)). We train the model with 35 days of rolling-window data, and evaluate on the 36th day data. The model is retrained from scratch everyday with most recent data. There are 31 categorical features and 30 dense scalar features to describe the apps, such as app ID, app title, user language, etc. The vocabulary size of the categorical features varies from hundreds to millions.

4.1.2 Evaluation Metrics. We will evaluate the model performance using AUC (Area Under the Receiver Operating Characteristic Curve). The AUC metric can be interpreted as the probability of the model assigning a randomly chosen positive example a higher ranking than a randomly chosen negative example. In the web-scale application dataset, due to the large size of the evaluation dataset, we notice that the improvement of AUC at 0.001 level achieves statistical significance. Similar observations have been made in other web-scale applications [3, 8, 15]. Additionally, we report the normalized LogLoss with Transformer one layer as the baseline.

We train the model on TPU [16, 17], and we report the training Queries per second (QPS) to measure the training speed. We estimate the latency through offline simulation, where we perform inference of the models on 20 batches of examples with batch size being 1024. We then normalize the latency of all models with respect to the baseline model, which is a one-layer Transformer model with pruning. Due to the large number of online serving requests, the serving latency is much more critical than training QPS.

4.1.3 Model Architectures. Note that our work is on feature interaction learning with Transformers, which is drastically different from user history sequence modeling with Transformers [20, 41]. Therefore, we focus on comparing our proposed models with the following SOTA feature interaction models:

- **AutoInt** [40] is proposed to learn feature interaction through the multi-head self-attention layer, and then learn the final predictions through a Multi-layer Perceptrons (MLP) layers.
- **DLRM** [29] is to learn feature interaction through factorization machine, and then learn implicit feature interactions through a MLP layer, for the final predictions.
- **DCN** [45] (*i.e.,* DCN-v2) is one of the SOTA models on feature interaction learning in web-scale application, which has a cross network that explicitly creates bounded-degree feature crosses where the feature interaction order increases with layer depth.
- **Transformer** [44] with multi-head self-attention layer for feature interaction learning. The main difference between Transformer and AutoInt is: 1. Transformer use encoded task embeddings for predictions; while AutoInt uses all encoded feature and task embeddings; 2. There is FFN layer in Transformer architecture, but not in the AutoInt layer.
- **Transformer + PE** is to add **P**osition **E**ncoding to the Transformer architecture, where we learn an additional per-feature embedding as feature encoding.
- **HeteroAtt** is our proposed Heterogeneous Attention layer to capture the heterogeneous feature interactions.
- **Hiformer** is our proposed model with low-rank approximation and model pruning.

4.1.4 Implementation Details. The model training and serving is built on Tensorflow 2 [38] and Model Garden [48]. To ensure a fair comparison, we keep all the model components, except for the feature interaction layers, to be the same. In the embedding layer, we set the embedding dimensions to be 128 for all models. Note that for Transformer architectures, HeteroAtt, and Hiformer , we only use the encoded CLS token (i.e., learned task token) for the final task predictions.

In our experiments on Transformers, we set model dimensionality $d = 128$, number of heads in the attention layer $H = 4$, and the layer size of FFN to be $d_f = 512$. We set $d_k = 16$ and $d_v = 64$, instead of setting $d_k = d/H, d_v = d/H$ as in the Transform model and the other Transformer-based models. For the other methods, we perform a hyper-parameters tuning to get the best performance. For online experiments, we tune the hyperparameters based on one day's (say Feb 1st) data, and fix the settings for the following days (i.e., dates after Feb 2nd).

4.2 Homogeneous vs Heterogeneous Feature Interaction (Q1)

First of all, we observe that Transformer+PE model performs similarly to the vanilla Transformer model. This is consistent with our hypothesis. Though coming with more parameters, the feature encoding only learns a bias embedding term per feature, which is not sufficient to learn the feature transformation for heterogeneous feature interaction learning. Secondly, we observe that Transformer model with heterogeneous attention layer (HeteroAtt) performs significantly better than the vanilla Transformer model. This result validates our argument that HeteroAtt can effectively capture the complex feature interactions through the transformation matrices $\mathbf{M}_{i,j}$. Moreover, we observe that the parameter # of the two-layer HeteroAtt model is much larger than the two-layer Transformer model, as the HeteroAtt model has more model expressiveness. Therefore, we can answer affirmatively to Q1, that the heterogeneous feature interaction with better feature context awareness is crucial for providing personalized recommendations.

4.3 Model Performance Comparison (Q2)

The offline comparison of the baseline models and our proposed models are summarized in Table 1. We report the best performance of each model, with the corresponding number of layers.

We take a closer look at the AutoInt, DLRM, and vanilla Transformer model. In the feature interaction learning of these models, there is no feature awareness and semantic alignment. Thus, their performance are relatively weaker compared with the other models. In DCN model, the Cross Net implicitly generates all the pairwise crosses and then projects it to a lower-dimensional space. The parameterization of all pairwise crosses are different, which is similar to providing feature awareness. Therefore, DCN model performance is comparable with our HeteroAtt models.

Compared with the heterogeneous attention layer (i.e., HeteroAtt), Hiformer model provides more model expressiveness in the QKV projections. It achieves the best model performance with just one layer, outperforming the HeteroAtt model and the other SOTA models.

4.4 Serving Efficiency (Q3)

We compare the serving efficiency of all the models. First of all, we compare the Transformer two layer model with the one layer model. As we can see, the latency of the two layer model is more than 2x of the one layer model. This is due to the fact that we only apply pruning to the second layer in the two layer model, and we have already applied pruning to the one layer model. Similarly, we can draw the same conclusion for the HeteroAtt layer as the two models have the same operators, thus latency. However, such pruning technique cannot be applied to AutoInt model. Therefore, the one-layer AutoInt model is much more expensive than the vanilla Transformer model and the HeteroAtt model.

Secondly, we compare the Hiformer model with the one-layer HeteroAtt model. Both the one-layer Hiformer model and the one-layer HeteroAtt model enjoy the inference optimization from pruning. However, because of the more expressive QKV projection, we see that the Hiformer model is 50.05% more expensive.

Model	Layer #	Parameter #	AUC (↑)	LogLoss (%, ↓)	Train QPS (↑)	Serving Latency (↓)
AutoInt	1	12.39M	0.7813	-0.37096	5.45e6	2.28
DLRM	-	5.95M	0.7819	-0.47695	5.14e6	0.95
DCN	1	13.73M	0.7857	-0.79491	5.76e6	1.46
Transformer	1	0.74M	0.7795	0	5.75e6	1.00
	2	0.84M	0.7811	-0.31797	4.12e6	3.03
	3	0.97M	0.7838	-0.45045	3.13e6	5.05
Transformer+PE	3	1.08M	0.7833	-0.39746	3.12e6	5.06
HeteroAtt (ours)	1	2.36M	0.7796	-0.05299	5.71e6	1.01
	2	10.50M	0.7856	-0.82141	4.10e6	3.11
Hiformer (ours)	1	16.68M	0.7875	-0.87440	5.69e6	1.52

Table 1: Offline model performance comparison regarding number of parameters, AUC, and normalized efficiency. The number of parameters excludes the embedding layer. ↑ (↓) means the higher (lower) the better.

Nevertheless, we want to point out that without low-rank approximation, the Hiformer model would potentially be much more expensive. In our experiment, we set rank of query and key as $r_k = 128$, and value $r_v = 1024$. We compare the Hiformer with and without low-rank approximation in Table 2. The low-rank approximation gives 62.7% inference latency saving. Additionally, there is no significant model quality loss, which is expected because of the observed low-rank structures of Hiformer query, key, and value projection matrices (*e.g.*, $\mathcal{V}$, shown in Figure 3).

Hiformer Model	Parameter #	AUC	Latency
low-rank approx.	16.68M	0.7875	1.52
w/o low-rank approx.	59.95M	0.7882	3.35

Table 2: Low-rank approximation for Hiformer .

4.5 Parameter Sensitivity (Q4)

Since one of the challenges is to reduce the serving latency, we are also interested in the parameter sensitivity of the model as in RQ4. More formally, we aim to answer the following question:

As we previously discussed in Section 4, instead of directly using $d_k = d/H$ and $d_v = d/H$, we find that the choice of d_k and d_v presents certain trade-offs between model performance and serving latency for the HeteroAtt and Hiformer model.

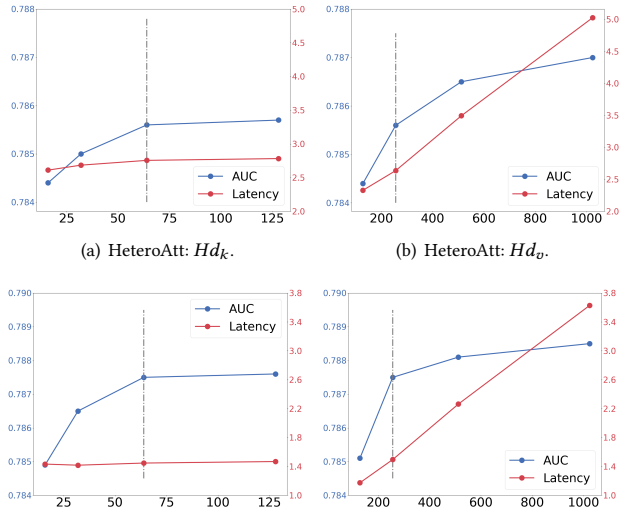
Firstly, for Hd_k in HeteroAtt in Figure 5(a), after reducing the Hd_k from $d = 128$ to 64, there is no model quality loss. However, we can harvest a free latency improvement (3%). If we further decreasing Hd_k, there is considerable model quality loss. Therefore, we opt for $Hd_k = 64$, as highlighted by the grey line. By comparison, increasing Hd_k for HeteroAtt model gives model quality improvement, but also significant latency increasing (in Figure 5(b)). For quality and latency trade-off, we set $Hd_v = 256$.

Similarly, we observe in Figure 5(c), for Hiformer model, decreasing Hd_k to 64 gives no quality loss, but unlike HeteroAtt model, there is almost no latency gain. This is due to the low-rank approximation in the QK projection, especially we set the rank $r_k = 256$, which is relatively small, compared with $r_v = 1024$. Accordingly, the query projection dominates the computation cost. With increased Hd_v, the latency increases significantly, shown in Figure 5(d).

This observation on the model quality and latency trade-off of Hd_v provides uniqueness of the HeteroAtt and Hiformer model. It can serves as a instrument to tune the model capacity at the cost of latency. For example, when the number of requests per query is relatively small, we can achieve better model performance by increasing d_v. Such an instrument might not not available for the other SOTA models, such as DCN [46].

4.6 Online A/B Testing

In addition to the extensive offline experiments, we also have conducted an online A/B testing experiment with the proposed model framework. For each of the control and treatment groups, there is 1% randomly selected users, who receive the item recommendations based on different ranking algorithms. To understand the improvement we gain through heterogeneous feature interaction learning, the baseline model is leveraging a one-layer Transformer model to capture feature interactions. We collect the users' engagement metrics for 10 days, as shown in Table 3. The one layer HeteroAtt model significantly outperforms the one layer Transformer model, which is consistent with our offline analysis. Additionally, two



(a) HeteroAtt: Hd_k. (b) HeteroAtt: Hd_v.

(c) Hiformer : Hd_k. (d) Hiformer : Hd_v.

Figure 5: Parameter Sensitivity. With increased d_k and d_v, the model quality gain saturates; however, the model inference cost increases drastically. The black dotted lines mark the selected d_k, d_v values, which gives the best quality and inference cost trade-off.

layer HeteroAtt model further improves over the one-layer one. The Hiformer performs the best among all models, including the DCN model. Such an observation indicates that Transformer-based architectures can outperform SOTA feature interaction models. We have successfully deployed the Hiformer model to production.

Model	Layer Num	Engagement Metrics
Transformer	1	+0.00%
HeteroAtt (ours)	1	**+1.27%***
HeteroAtt (ours)	2	**+2.33%***
Hiformer (ours)	1	**+2.66%***
DCN	1	**+2.20%***

Table 3: The online engagement and serving latency metrics of different models. * indicates statistically significant.

5 RELATED WORK

Before the era of deep learning, people often manually crafted cross features and added them to logistic regression models to improve their performance. Some also leveraged decision tree based models [11] to learn feature crosses. Later on, the development of embedding techniques have led to the design of Factorization Machines (FMs) [34], which was proposed to model second-order feature interactions with the inner product of two latent vectors.

Recently, DNNs have become the backbone of many models in industry. To improve the efficiency of feature cross learning, many works explicitly model feature interactions by designing a function $g(x_i, x_j)$ while leveraging the implicit feature crosses learned by DNNs. Wide & deep model [3] combines a DNN model with a wide component that consists of crosses of raw features. Since then, many work has been proposed to automate the manual feature cross work of wide & deep by introducing FM-like operations (inner products) or Hadamard products. DeepFM [8] and DLRM [29] adopt FM in the model, Neural FM [10] generalizes FM by incorporating the Hadamard product, PNN [32] also uses inner products and outer products to capture pairwise feature interactions. These methods however can only model up to second order feature crosses. There is also work that can model higher-order feature interactions. Deep Cross Network (DCN) [45, 46] designs a cross network to explicitly learn high-order feature interactions where its interaction order increases with layer depth. DCN-V2 [46] makes DCN more expressive and practical in large-scale industrial settings. xDeepFM [26] also improves the expressiveness of cross network in DCN and relies on the Hadamard product to capture high-order feature crosses.

Another line of work leverages the attention mechanism in Transformer models to capture feature interactions. AutoInt [40] was proposed to capture the feature interaction through the multi-head self-attention layer. Nevertheless, as the Transformer was designed to model relationships that are context independent, such as text tokens, directly applying the multi-head self-attention layer for feature interaction has very limited model expressiveness. Similarly, [25] is based on the attention mechanism with cross-product transformation to capture hierarchical attention, without providing feature awareness and semantic alignment.

The Heterogeneous Graph Transformer [13] work is related to ours, where node types are considered in the heterogeneous mutual attention layer. Though we share some similarities in the motivations, our work is substantially different because Hiformer model is proposed to capture feature interactions for web-scale recommender system, and the design of the Hiformer attention layer has more expressiveness. The field-aware factorization machine [18] work is also relevant. In [18], the feature awareness of feature interaction learning is implemented through individual embedding lookup tables, while our proposed methods are through the information transformation in the heterogeneous attention layer. PNN [33] leverages kernel Factorization Machine to consider feature dynamics in feature interaction learning, which shares similarity with the heterogeneous attention layer. Nevertheless, heterogeneous attention layer is attention based while PNN is derived from factorization machine. Additionally, we leverage Query and Key projection to learn heterogeneous feature interactions while PNN is based on kernel matrices. Moreover, we further increase model expressiveness in Hiformer with cost reduction.

6 CONCLUSION AND FUTURE WORK

In this work, we propose a heterogeneous attention layer, which is a simple but effective modification of the self-attention layer to provide feature awareness for feature interaction learning. We then further improve the model expressiveness, and propose Hiformer to improve feature interaction learning. We also improved the serving efficiency of Hiformer such that it could satisfy the serving latency requirement for web-scale applications. The key change in our method is to identify the different relationships among different features in the attention mechanism, such that we can capture semantic relevance given the dynamic information in learned feature embeddings. Both offline and online experiments on the world leading digital distribution service platform demonstrate the effectiveness and efficiency of our method. These experiment results indicate that a Transformer based model for feature interaction learning can perform better than SOTA methods in web-scale applications.

Our work serves as a bridge to bring Transformer-based architectures to applications in web-scale recommender system. For future work, we are interested in how the recent advances in Transformer architectures [9] in other domains domains, such as NLP and CVR, can be translated to recommender systems. With the new feature interaction component introduced in this paper, it can also be leveraged in neural architecture search [7] to further improve recommender systems.

REFERENCES

[1] Beutel, A., Covington, P., Jain, S., Xu, C., Li, J., Gatto, V., and Chi, E. H. Latent cross: Making use of context in recurrent recommender systems. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining* (2018), pp. 46–54.

[2] Caruana, R. Multitask learning. *Machine learning 28*, 1 (1997), 41–75.

[3] Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., et al. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems* (2016), pp. 7–10.

[4] Covington, P., Adams, J., and Sargin, E. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems* (2016), pp. 191–198.

[5] Davidson, J., Liebald, B., Liu, J., Nandy, P., Van Vleet, T., Gargi, U., Gupta, S., He, Y., Lambert, M., Livingston, B., et al. The youtube video recommendation system. In *Proceedings of the fourth ACM conference on Recommender systems* (2010), pp. 293–296.

[6] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[7] Elsken, T., Metzen, J. H., and Hutter, F. Neural architecture search: A survey. *The Journal of Machine Learning Research 20*, 1 (2019), 1997–2017.

[8] Guo, H., Tang, R., Ye, Y., Li, Z., and He, X. Deepfm: a factorization-machine based neural network for ctr prediction. *arXiv preprint arXiv:1703.04247* (2017).

[9] Han, K., Wang, Y., Chen, H., Chen, X., Guo, J., Liu, Z., Tang, Y., Xiao, A., Xu, C., Xu, Y., et al. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence* (2022).

[10] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web* (2017), pp. 173–182.

[11] He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., et al. Practical lessons from predicting clicks on ads at facebook. In *Proceedings of the eighth international workshop on data mining for online advertising* (2014), pp. 1–9.

[12] Hendrycks, D., and Gimpel, K. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).

[13] Hu, Z., Dong, Y., Wang, K., and Sun, Y. Heterogeneous graph transformer. In *Proceedings of the web conference 2020* (2020), pp. 2704–2710.

[14] Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., and Carreira, J. Perceiver: General perception with iterative attention. In *International conference on machine learning* (2021), PMLR, pp. 4651–4664.

[15] Joglekar, M. R., Li, C., Chen, M., Xu, T., Wang, X., Adams, J. K., Khaitan, P., Liu, J., and Le, Q. V. Neural input search for large scale recommendation models. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2020), pp. 2387–2397.

[16] Jouppi, N., Young, C., Patil, N., and Patterson, D. Motivation for and evaluation of the first tensor processing unit. *ieee Micro 38*, 3 (2018), 10–19.

[17] Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture* (2017), pp. 1–12.

[18] Juan, Y., Zhuang, Y., Chin, W.-S., and Lin, C.-J. Field-aware factorization machines for ctr prediction. In *Proceedings of the 10th ACM conference on recommender systems* (2016), pp. 43–50.

[19] Kalyan, K. S., Rajasekharan, A., and Sangeetha, S. Ammus: A survey of transformer-based pretrained models in natural language processing. *arXiv preprint arXiv:2108.05542* (2021).

[20] Kang, W.-C., and McAuley, J. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)* (2018), IEEE, pp. 197–206.

[21] Khan, S., Naseer, M., Hayat, M., Zamir, S. W., Khan, F. S., and Shah, M. Transformers in vision: A survey. *ACM computing surveys (CSUR) 54*, 10s (2022), 1–41.

[22] LeCun, Y., Bengio, Y., and Hinton, G. Deep learning. *nature 521*, 7553 (2015), 436–444.

[23] Li, D., Hu, B., Chen, Q., Wang, X., Qi, Q., Wang, L., and Liu, H. Attentive capsule network for click-through rate and conversion rate prediction in online advertising. *Knowledge-Based Systems 211* (2021), 106522.

[24] Li, M., Liu, Z., Smola, A. J., and Wang, Y.-X. Difacto: Distributed factorization machines. In *Proceedings of the Ninth ACM International Conference on Web Search and Data Mining* (2016), pp. 377–386.

[25] Li, Z., Cheng, W., Chen, Y., Chen, H., and Wang, W. Interpretable click-through rate prediction through hierarchical attention. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (2020), pp. 313–321.

[26] Lian, J., Zhou, X., Zhang, F., Chen, Z., Xie, X., and Sun, G. xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (2018), pp. 1754–1763.

[27] Lu, J., Wu, D., Mao, M., Wang, W., and Zhang, G. Recommender system application developments: a survey. *Decision Support Systems 74* (2015), 12–32.

[28] Mirhoseini, A., Goldie, A., Yazgan, M., Jiang, J. W., Songhori, E., Wang, S., Lee, Y.-J., Johnson, E., Pathak, O., Nazi, A., et al. A graph placement methodology for fast chip design. *Nature 594*, 7862 (2021), 207–212.

[29] Naumov, M., Mudigere, D., Shi, H.-J. M., Huang, J., Sundaraman, N., Park, J., Wang, X., Gupta, U., Wu, C.-J., Azzolini, A. G., et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).

[30] Okura, S., Tagami, Y., Ono, S., and Tajima, A. Embedding-based news recommendation for millions of users. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (2017), pp. 1933–1942.

[31] Pan, X., Li, M., Zhang, J., Yu, K., Wen, H., Wang, L., Mao, C., and Cao, B. Metacvr: Conversion rate prediction via meta learning in small-scale recommendation scenarios. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2022), pp. 2110–2114.

[32] Qu, Y., Cai, H., Ren, K., Zhang, W., Yu, Y., Wen, Y., and Wang, J. Product-based neural networks for user response prediction. In *2016 IEEE 16th International Conference on Data Mining (ICDM)* (2016), IEEE, pp. 1149–1154.

[33] Qu, Y., Fang, B., Zhang, W., Tang, R., Niu, M., Guo, H., Yu, Y., and He, X. Product-based neural networks for user response prediction over multi-field categorical data. *ACM Transactions on Information Systems (TOIS) 37*, 1 (2018), 1–35.

[34] Rendle, S. Factorization machines. In *2010 IEEE International conference on data mining* (2010), IEEE, pp. 995–1000.

[35] Resnick, P., and Varian, H. R. Recommender systems. *Communications of the ACM 40*, 3 (1997), 56–58.

[36] Schafer, J. B., Konstan, J., and Riedl, J. Recommender systems in e-commerce. In *Proceedings of the 1st ACM conference on Electronic commerce* (1999), pp. 158–166.

[37] Shazeer, N. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202* (2020).

[38] Singh, P., Manure, A., Singh, P., and Manure, A. Introduction to tensorflow 2.0. *Learn TensorFlow 2.0: Implement Machine Learning and Deep Learning Models with Python* (2020), 1–24.

[39] Song, Q., Cheng, D., Zhou, H., Yang, J., Tian, Y., and Hu, X. Towards automated neural interaction discovery for click-through rate prediction. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2020), pp. 945–955.

[40] Song, W., Shi, C., Xiao, Z., Duan, Z., Xu, Y., Zhang, M., and Tang, J. Autoint: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (2019), pp. 1161–1170.

[41] Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, P. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management* (2019), pp. 1441–1450.

[42] Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. Efficient transformers: A survey. *ACM Computing Surveys 55*, 6 (2022), 1–28.

[43] Tsang, M., Cheng, D., and Liu, Y. Detecting statistical interactions from neural network weights. *arXiv preprint arXiv:1705.04977* (2017).

[44] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems 30* (2017).

[45] Wang, R., Fu, B., Fu, G., and Wang, M. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 2017, pp. 1–7.

[46] Wang, R., Shivanna, R., Cheng, D., Jain, S., Lin, D., Hong, L., and Chi, E. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the Web Conference 2021* (2021), pp. 1785–1797.

[47] Yang, J., Yi, X., Zhiyuan Cheng, D., Hong, L., Li, Y., Xiaoming Wang, S., Xu, T., and Chi, E. H. Mixed negative sampling for learning two-tower neural networks in recommendations. In *Companion Proceedings of the Web Conference 2020* (2020), pp. 441–447.

[48] Yu, H., Chen, C., Du, X., Li, Y., Rashwan, A., Hou, L., Jin, P., Yang, F., Liu, F., Kim, J., et al. Tensorflow model garden. *GitHub* (2020).

[49] Zhang, S., Yao, L., Sun, A., and Tay, Y. Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys (CSUR) 52*, 1 (2019), 1–38.

[50] Zhao, Z., Hong, L., Wei, L., Chen, J., Nath, A., Andrews, S., Kumthekar, A., Sathiamoorthy, M., Yi, X., and Chi, E. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM Conference on Recommender Systems* (2019), pp. 43–51.

[51] Zhuang, H., Wang, X., Bendersky, M., and Najork, M. Feature transformation for neural ranking models. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (2020), pp. 1649–1652.